Reviewer Pack — Minimal Index of Topological Entanglement and Communication …

Entry fa016e1f9493 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 17:11:09 UTC

Minimal Index of Topological Entanglement and Communication Complexity Rank Variance Ratio

Entry ID: `fa016e1f9493` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-08 17:11:09 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Topological Entanglement) **Field B** (complexity object): Communication Complexity (Matrix Rank Variance)

Statement:

> For all n-variables boolean functions f, the minimal index of topological entanglement μ(f) is linearly correlated with the communication complexity rank variance ratio R(f), such that μ(f) = Θ(R(f)) and R(f) = Ω(μ(f)).

Rationale (proposer's reasoning):

> Topological entanglement measures quantum correlations and its study in quantum information theory could potentially reveal underlying structures in classical communication complexity. A higher topological entanglement suggests complex correlations, which might correspond to larger rank variances in communication complexity, leading to a deeper understanding of the interplay between quantum and classical computational tasks.

Taxonomy category: `TOPENTANGLED_TO_COMM Rank Var Ratio` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `43acbdc3d3f061fa`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For all n-variables boolean functions f, if the correlation coefficient r between μ(f) and R(f) is ≥ 0.7 for at least 80% of the 30 seeds, then support the conjecture; otherwise, falsify it.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def calculate_minimal_index_of_topological_entanglement(f):
        # Placeholder implementation
        return random.random()

    def calculate_communication_complexity_rank_variance(f):
        # Placeholder implementation
        return random.random()

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        f = generate_boolean_function(n)
        mu_f = calculate_minimal_index_of_topological_entanglement(f)
        R_f = calculate_communication_complexity_rank_variance(f)
        results.append({"n": n, "mu_f": mu_f, "R_f": R_f})

    if len(results) < 30:
        return {
            "metric_name": "correlation_coefficient",
            "metric_value": None,
            "instances_tested": len(results),
            "n_max": max(r["n"] for r in results),
            "conjecture_holds": False,
            "counterexample": "not_enough_instances"
        }
```

```
    mu_values = [r["mu_f"] for r in results]
    R_values = [r["R_f"] for r in results]

    mean_mu = sum(mu_values) / len(mu_values)
    mean_R = sum(R_values) / len(R_values)

    covariance = sum((mu_values[i] - mean_mu) * (R_values[i] - mean_R) for i in range(len(mu_value
    variance_mu = sum((mu_values[i] - mean_mu)**2 for i in range(len(mu_values))) / len(mu_values)
    variance_R = sum((R_values[i] - mean_R)**2 for i in range(len(R_values))) / len(R_values)

    correlation_coefficient = covariance / (math.sqrt(variance_mu) * math.sqrt(variance_R))

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": 30,
        "n_max": max(r["n"] for r in results),
        "conjecture_holds": correlation_coefficient >= 0.7,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    if all(r["metric_value"] is not None for r in results):
        mean_metric = sum(r["metric_value"] for r in results) / len(results)
        std_metric = math.sqrt(sum((r["metric_value"] - mean_metric)**2 for r in results) / len(re
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={suppor
        else:
            first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result['
            print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}'
    else:
        print("RESULT: INCONCLUSIVE some_metric_values_missing")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means it did not complete its execution to determine the correlation coefficient and support fraction. | next: Re-run the test with increased time limits or optimize the code to ensure completion within the allocated time.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14998
2	preregistration	ollama_remote	glm4:latest	0	0	9319
3	novelty	ollama_remote	glm4:latest	0	0	8591
4	novelty	ollama_remote	glm4:latest	0	0	8698
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	39760
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12660
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10025
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11853
9	judge	ollama_remote	glm4:latest	0	0	42686

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 158590 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/fa016e1f9493.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/fa016e1f9493.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/fa016e1f9493.tar.gz` (if generated)